Supplement: Cost Decreases Trigger Tight Fibonacci Cascades in Saturated Cost Partitioning

Haoyi Zhang[a], Huaijin Ran[b], Yuqiang Sun[b], Jiongchi Yu[b], Yang Liu[b,*]

[a] *Xi'an Jiaotong-Liverpool University, Suzhou, China*
[b] *Nanyang Technological University, Singapore*

Abstract

This artifact presents a finite labelled model with fixed goal-reachable abstractions, a fixed order, and finite nonnegative label costs. It gives the saturation invariant, path-relay proof, tight support-two Fibonacci bound and matching construction, exact table-reuse criterion, ordered-repair proof, and counterexamples. Exact integer and rational checks cover implementation behavior and concrete witnesses; the evaluation protocol documents the bounded synthetic measurements. A 65-source literature audit records each work's role, comparison boundary, verification source, and reading depth: 15 substantive full-paper or theorem/algorithm passes and 50 bibliographic and relevant-passage checks.

Model and the Invariant Being Preserved

A concrete task is a finite labelled transition system with goal set G and one finite nonnegative cost $c(a)$ for every action label $a \in A$. An abstraction map α_i preserves every labelled transition and maps concrete goals to abstract goals. We fix these maps, their finite graphs $D_i = (V_i, E_i, G_i)$, and one order $1, \ldots, m$. Every abstract vertex can reach a goal. Costs may change only between searches. This is an explicit finite-distance model: the implementation rejects unreachable abstract vertices rather than assigning a meaning to arithmetic with infinity.

*Corresponding author: Yang Liu.

Email addresses: hyeliozhang@gmail.com (Haoyi Zhang), huaijin003@e.ntu.edu.sg (Huaijin Ran), yuqiang.sun@ntu.edu.sg (Yuqiang Sun), jiongchiyu@acm.org (Jiongchi Yu), yangliu@ntu.edu.sg (Yang Liu)

For an incoming residual $r_i \geq 0$, let h_i be its exact abstract goal distance and put

$$s_i(a) = \max\big(\{0\} \cup \{h_i(u) - h_i(v) : (u, v, a) \in E_i\}\big),$$

$$r_{i+1} = r_i - s_i.$$

An absent label and a label with only self-loops receive saturation zero. Several transitions may share one label; their costs are not independently mutable. Throughout, label–pattern incidence counts only non-self-loop transitions. We use nonnegative saturation, whereas the general SCP literature also permits negative allocations [55].

Lemma 1 (standard saturation invariant). For each stage, $0 \leq s_i \leq r_i$, and $h_i = \mathrm{dist}_{s_i}$. Consequently, the projected sum $H(s) = \sum_i h_i(\alpha_i(s))$ is consistent and admissible for the concrete costs c.

Proof. Exact distances satisfy $h_i(u) \leq r_i(a) + h_i(v)$, and are zero at goals. Taking all differences for a label gives $s_i \leq r_i$. Every edge also satisfies $h_i(u) \leq s_i(a) + h_i(v)$. Telescoping any goal path therefore gives $h_i \leq \mathrm{dist}_{s_i}$. Since $s_i \leq r_i$, monotonicity of path costs gives $\mathrm{dist}_{s_i} \leq \mathrm{dist}_{r_i} = h_i$. Thus equality holds. Subtraction preserves a nonnegative residual and implies $\sum_i s_i(a) \leq c(a)$. Project a concrete edge, sum its abstract consistency inequalities, and obtain $H(s) \leq c(a) + H(t)$. Concrete goals have value zero, so telescoping proves admissibility. This is existing cost-partitioning reasoning, included to state precisely what the update maintains. □

Arbitrarily Long Propagation on an Incidence Path

Use Boolean variables $x_1, \ldots, x_m$, goal all ones, and their singleton projections in that order. For $j = 0, \ldots, m$, action a_j sets x_j, x_{j+1} to one when those indices lie in $1, \ldots, m$; it has no precondition. Thus a_0 only affects x_1 and a_m only affects x_m. Costs are $c(a_0) = 1$ and $c(a_j) = 2$ otherwise. In pattern i, advancing labels are a_{i-1}, a_i; all other transitions are self-loops. An advancing label also has a self-loop at the goal. Each action affects at most two patterns, and the incidence graph is a path.

Proposition 2 (path relay). Replacing $c(a_0)$ by $1 - \delta$, for $0 < \delta \leq 1$, changes every non-goal table entry from 1 to $1 + (-1)^i\delta$. After stage i, the only residual difference is on label a_i, and equals $(-1)^{i+1}\delta$.

Proof. Before old stage i, label a_{i-1} has residual 1 and fresh label a_i has residual 2. The first statement is immediate for $i = 1$. If the only incoming difference is $(-1)^i\delta$ on a_{i-1}, its new cost is in $[0, 2]$. The non-goal distance, the minimum of these two advancing costs, is therefore $1 + (-1)^i\delta$, even

at the boundary tie when it equals 2. Both advancing labels receive this saturation. Label a_{i-1} is exhausted in both partitions; the new residual of a_i is $1-(-1)^i\delta$. All other allocations are zero, so their incoming differences remain zero. This proves the induction and the original-cost formula. □

The original changed label has non-self-loop transitions in only the first pattern. Sparsity of its original incidence, maximum degree two, and residual support of size one therefore do not bound the number of changed tables independently of m. Any representation that explicitly overwrites every changed table cell needs at least m writes on this family. This statement concerns materialized cells, not lazy formulas, shared symbolic representations, or all possible update algorithms.

Amplification with Bounded Incidence

For a Boolean singleton pattern with nonempty advancing set B, all other labels are self-loops. Its non-goal distance is $\min_{a\in B} r(a)$, and that value is saturated on every advancing label. Let T_B denote the resulting residual map.

Lemma 3 (one-table perturbation bound). For all finite nonnegative residuals, $\|T_B(r') - T_B(r)\|_\infty \le 2\|r' - r\|_\infty$.

Proof. Write $\eta = \|r' - r\|_\infty$ and $m = \min_{a\in B} r(a)$, $m' = \min_{a\in B} r'(a)$. The minimum is 1-Lipschitz, so $|m' - m| \le \eta$. Outside B, T_B leaves a coordinate unchanged. Inside B, the output difference is $(r'(a) - r(a)) - (m' - m)$, whose absolute value is at most 2η. □

The factor-two bound permits growth under composition but does not identify when it can begin. The unit perturbation has a sharper two-stage bound.

Lemma 4 (tight amplification onset). Suppose r' is obtained from r by decreasing one coordinate by one. For any two nonempty advancing sets B, C for which the residuals are defined,

$$\|T_C(T_B(r')) - T_C(T_B(r))\|_\infty \le 1.$$

There is a three-table sequence whose final residual difference has magnitude 2.

Proof. Let $\mu = \min_B r' - \min_B r \in [-1, 0]$. If the changed coordinate is in B, the first output differences are $-1-\mu$ there, $-\mu$ on the other labels of B, and zero elsewhere; otherwise they are simply -1 and zero. Thus all first-stage differences lie in an interval $[\ell, u]$ of width one that contains zero. At the second table, the change ν of the selected minimum lies in $[\ell, u]$.

Selected output differences have the form $d - \nu \in [-1, 1]$; unselected ones already lie in $[-1, 1]$. The construction below with $k = 0$ attains magnitude 2 after its third table. □

For a vector d of support at most two, put $M(d) = \|d\|_\infty$ and $L(d) = \|d\|_1$. Call d type P if its nonzero coordinates have one sign (a singleton and the zero vector count as P), and type N if it has one positive and one negative coordinate.

Lemma 5 (one-step support-two bounds). Let $d = r' - r$ and $e = T_B(r') - T_B(r)$, and suppose both have support at most two. Then

input	output	$M(e)$	$L(e)$
P	P	$\le M(d)$	$\le 2M(d)$
P	N	$\le M(d)$	$\le L(d)$
N	P	$\le L(d)$	$\le L(d) + M(d)$
N	N	$\le L(d)$	$\le L(d)$

Proof. Write $\mu = \min_B r' - \min_B r$. The elementary minimum inequality gives

$$\min_{a \in B} d(a) \le \mu \le \max_{a \in B} d(a),$$

$$e(a) = \begin{cases} d(a) - \mu, & a \in B, \\ d(a), & a \notin B. \end{cases}$$

The statement is invariant under exchanging r, r' and permuting labels. For type P, assume the at most two nonzero entries are nonnegative. Then $\mu \ge 0$ whenever a changed entry is selected. No output magnitude exceeds $M(d)$. If both signs survive, positive mass can only decrease; the single negative magnitude is no larger than the decrease of a selected entry attaining at least μ. Hence $L(e) \le L(d)$. If one sign survives, support two gives $L(e) \le 2M(d)$.

For type N, write the entries as a and $-b$, with $a, b > 0$. Since $-b \le \mu \le a$, every possible output entry—a, $-b$, $a - \mu$, $-b - \mu$, or $-\mu$—has magnitude at most $a + b = L(d)$. If both signs survive, their separation is at most $a + b$: when $\mu > 0$, selecting the negative endpoint forces the positive endpoint to be selected as well, and symmetrically for $\mu < 0$. Thus $L(e) \le L(d)$. If only one sign survives, at most one entry can be derived from an old nonzero endpoint with magnitude at most $L(d)$; a second survivor can only be a shifted zero of magnitude $|\mu| \le M(d)$. Therefore $L(e) \le L(d) + M(d)$. □

Write $F_1 = F_2 = 1$ for the Fibonacci numbers.

Theorem 6 (tight support-two envelope). Start from two nonnegative residual vectors that differ by decreasing one coordinate by one. Apply

any sequence of nonempty two-state minimum-subtraction maps, and assume the residual-difference support is at most two after every map. After $2k+1$ maps,

$$\|d_{2k+1}\|_\infty \le F_{k+2} \qquad (k \ge 0).$$

In particular, after $2k+3$ maps the bound is F_{k+3}.

Proof. We prove the following stronger parity invariants. After $2k+1$ maps,

	M	L	$L+M$
P	$\le F_{k+2}$	$\le F_{k+3}$	$--$
N	$\le F_{k+2}$	$2L \le F_{k+4}$	$2(L+M) \le F_{k+5}$,

(1)

and after $2k+2$ maps,

	M	L	$L+M$
P	$2M \le F_{k+4}$	$2L \le F_{k+5}$	$--$
N	$--$	$L \le F_{k+3}$	$L+M \le F_{k+4}$.

(2)

After the first map, $M \le 1$. A type-P output has $L \le 2$, while a type-N output lies on opposite sides of zero in an interval of width one, so $L \le 1$; these facts establish (1) for $k = 0$.

Apply Lemma 5. From odd P to even P, $2M' \le 2F_{k+2} \le F_{k+4}$ and $2L' \le 4F_{k+2} \le F_{k+5}$; odd P to even N gives $L' \le F_{k+3}$ and $L'+M' \le F_{k+3}+F_{k+2} = F_{k+4}$. From odd N to even P, $2M' \le 2L \le F_{k+4}$ and $2L' \le 2(L+M) \le F_{k+5}$; odd N to even N gives $L' \le L \le F_{k+4}/2 \le F_{k+3}$ and $L'+M' \le 2L \le F_{k+4}$. This proves (2).

Conversely, even P to odd P gives $M' \le F_{k+4}/2 \le F_{k+3}$ and $L' \le F_{k+4}$; even P to odd N gives $M' \le F_{k+3}$, $2L' \le F_{k+5}$, and $2(L'+M') \le F_{k+5} + F_{k+4} = F_{k+6}$. Even N to odd P gives $M' \le F_{k+3}$ and $L' \le F_{k+4}$; even N to odd N gives $M' \le F_{k+3}$, $2L' \le 2F_{k+3} \le F_{k+5}$, and $2(L'+M') \le 4F_{k+3} \le F_{k+6}$. The scalar inequalities used here follow from the Fibonacci recurrence, monotonicity, $F_{n+1} \le 2F_n$, and $4F_n \le F_{n+3}$ for $n \ge 2$. Thus (1) and (2) hold by induction, and the P/N bounds in (1) both imply the theorem. □

Theorem 7 (tight bounded-incidence cascade). For every integer $k \ge 0$ there is a fixed Boolean task with $2k+3$ singleton patterns and $2k+4$ labels. One external cost change from 1 to 0 changes every table and attains the support-two upper bound: it leaves exactly one final residual difference of value F_{k+3}. Every action affects at most three variables, every pattern has at most three advancing actions, and every intermediate residual-difference vector has support at most two. The largest original cost is exactly F_{k+3}.

Construction and proof. For each pattern, a concrete action sets its mentioned Boolean variable to one and is a self-loop elsewhere. Begin with labels

d, x, y, original costs $(1, 1, 2)$, and advancing set $\{d, x, y\}$. Decreasing only d to zero changes the table value by -1 and leaves

$$r(x) = 0, \quad \Delta r(x) = 1, \qquad r(y) = 1, \quad \Delta r(y) = 1,$$

where r is the old residual and $\Delta r = r' - r$.

Maintain an active pair x, y satisfying

$$\begin{aligned} r(x) &= 0, \quad \Delta r(x) = A, \\ r(y) &= A, \quad \Delta r(y) = B, \qquad A \geq B > 0. \end{aligned}$$

Introduce fresh labels z, w with original costs A and $2A + B$, and append advancing sets

$$\{x, z\}, \qquad \{y, z, w\}.$$

At the first table, the old and new minima are 0 and A. Its value changes by $+A$; the only remaining differences are $\Delta r(y) = B$ and $\Delta r(z) = -A$. At the second table, the old and new minima are A and 0. Its value changes by $-A$ and leaves

$$\begin{aligned} r(y) &= 0, & \Delta r(y) &= A + B, \\ r(w) &= A + B, & \Delta r(w) &= A. \end{aligned}$$

Hence two tables map the carry pair (A, B) to $(A + B, A)$. Starting from $(1, 1)$, after k iterations the pair is (F_{k+2}, F_{k+1}).

For cleanup, introduce one fresh label z of cost A and append $\{x, z\}$ and $\{y, z\}$. Their value changes are $+A$ and $-A$, and the final residual has only $\Delta r(y) = A + B = F_{k+3}$. Thus all $1 + 2k + 2$ tables change and support never exceeds two. Each fresh z occurs twice. A carry label occurs in the table that creates it and in at most the next two tables; hence action incidence is at most three, while each advancing set has size at most three. There are $3 + 2k + 1$ labels. In iteration j, the new w costs $2F_{j+2} + F_{j+1} = F_{j+4}$, so the largest cost is the last such value, or 2 when $k = 0$, namely F_{k+3}. Theorem 6 proves that no support-two cascade can exceed this final magnitude at the same odd depth. □

All labels have goal self-loops in both the relay and amplifier. Their maximum edge difference is therefore already nonnegative. These witnesses also apply when the saturation formula is not explicitly clipped at zero. The general update implementation is nevertheless restricted to nonnegative residuals and allocations.

The amplification is exponential in the number of patterns, but it measures absolute residual sensitivity rather than optimal plan cost, search expansions, or running time. At the all-zero start, table-value changes -1 and

pairs $+A, -A$ sum to -1. Applying d and all fresh z labels covers every variable at a cost equal to the old heuristic sum; the same plan costs one less after the update. Admissibility thus proves that the old and new concrete optima equal those sums. At $k = 0$ and state $(1, 0, 1)$, the heuristic instead rises from zero to one, while the optimum stays one because x or z supplies the only missing variable for one under either cost vector.

Largest-cost bit width is $\lceil \log_2(F_{k+3} + 1) \rceil = \Theta(k)$, not constant. Every nonnegative residual coordinate stays below its original input cost; hence any two integer inputs with costs at most $2^b - 1$ have residual differences at most $2^b - 1$. The witness stores $O(k^2)$ cost bits and has $O(k^2)$ materialized edges including self-loops. Arithmetic must account for bit width. No upper bound is asserted once difference support can exceed two or for abstractions with more than two states.

The portable untimed proof-arithmetic runner checks the supplied extremal witnesses, their matching goal plans, and ten tiny concrete-optimum comparisons by exhaustive action-subset coverage. This is exact for nonnegative, commuting, idempotent bit-setting actions; it is not an optimum solver for general planning tasks. The artifact retains its exact-arithmetic and concrete-optimum results. The main note includes the complete core proof; these details and finite tests do not extend its domain.

An Exact Reuse Boundary and Ordered Repair

Let $h = \mathrm{dist}_r$ be an old exact, finite table in a fixed graph, s its nonnegative saturation, and $q \geq 0$ the new incoming residual. Define the new tight-edge graph by

$$E_q^h = \{(u, v, a) \in E : h(u) = q(a) + h(v)\}.$$

Proposition 8 (exact reuse). The entire old table remains exact under q if and only if (i) $q(a) \geq s(a)$ for every label and (ii) every vertex reaches a goal through E_q^h.

Proof. If $h = \mathrm{dist}_q$, every edge's consistency inequality gives $h(u) - h(v) \leq q(a)$; nonnegativity gives (i). Since the graph is finite and costs are nonnegative, any optimal goal walk has a cycle-free optimal goal path after removing cycles, possibly retaining zero-cost alternatives. Each edge on such a path is tight for exact goal distances: a strict inequality would give a cheaper suffix and contradict optimality. Thus (ii) holds. Conversely, (i) makes h consistent under q, so telescoping every goal path gives $h \leq \mathrm{dist}_q$. A tight goal path from any vertex telescopes to a path costing exactly h

there, giving $\text{dist}_q \leq h$. Goals have $h = 0$, including the zero-length path case. □

This is the standard shortest-path lower-bound and witness-path optimality criterion in shared-label form. The new tight graph is important: a formerly non-tight edge may become an essential alternative. It is not sufficient to preserve one old shortest-path tree. Label feasibility alone preserves admissibility of h under q, but does not imply equality with the new exact table.

Ordered repair. Start with residual c'. At each pattern, retain (h_i, s_i) exactly when Proposition 8 holds; otherwise compute exact distances with reverse Dijkstra and recompute saturation by scanning all labelled transitions. In either case, save the current incoming residual with the table and subtract the corresponding saturation from it before advancing.

Correctness. Inductively assume the incoming residual equals that of a full fresh SCP computation. A retained table is exact under that same residual by Proposition 8, so its saturation, a deterministic function of the table and graph, also equals the fresh saturation. A rebuilt table equals it by exact shortest-path computation. Subtraction therefore gives the same next residual. The induction proves equality of all distances, allocations, and offered residuals with full rebuilding, and Lemma 1 gives consistency and admissibility. This induction also covers a sequence of cost updates, provided each uses the latest valid cache and the same graph/order. It does not authorize arbitrary externally supplied tables or changes of abstraction identity.

The predicate skips every unchanged distance table; if it fails, at least one distance cell actually differs. This is minimal table rebuilding among schemes that retain whole old tables or rebuild whole tables. It is not an optimality statement about edge scans or time. A dynamic shortest-path algorithm could replace reverse Dijkstra inside a rebuilding stage without removing the need to propagate new residuals [48].

Implementation shortcuts and cost. Labels appearing only on self-loops have no distance effect and saturation zero. If all other incoming costs are unchanged, retain immediately. Otherwise, test feasibility only on changed effective labels; unchanged ones were already feasible. If these changes are all decreases, monotonicity gives $\text{dist}_q \leq h$, and feasibility gives the reverse inequality, so retain without a reachability scan. In the remaining case, reverse breadth-first traversal of the new tight edges from all goals checks reachability. Apart from labelled-cost comparisons, this takes $O(|V| + |E|)$ time and $O(|V|)$ auxiliary space. Full rebuilding uses reverse Dijkstra and one saturation scan. Scanning and rebuilding work are both

counted in the timing experiment.

Negative Controls and Admissible Baselines

A zero-tight component. With goal 2 and edges $0 \xrightarrow{a} 1$, $1 \xrightarrow{a} 0$, $0 \xrightarrow{b} 2$, old costs $(a, b) = (0, 1)$ give $h = (1, 1, 0)$ and $s = (0, 1)$. New costs $(0, 2)$ satisfy label feasibility. Each non-goal vertex still has a tight outgoing edge on the zero cycle, so local Bellman equality accepts this wrong table. The actual new distances are $(2, 2, 0)$. Reverse tight-goal reachability rejects it. Thus zero-cost SCCs cannot be treated as goal witnesses.

A newly tight alternative. Two parallel goal edges labelled a, b, with old costs $(1, 2)$ and new costs $(2, 1)$, keep distance 1 while the unique tight label switches. The exact predicate accepts this table. A second control starts from $(1, 1)$ and increases one label to 2; the surviving tie also preserves exactness.

Feasible new allocation, stale value. For the three-pattern path with old costs $(1, 2, 2, 2)$, decrease the first cost to zero. New non-goal table values are $(0, 2, 0)$, and the new allocations sum within $(0, 2, 2, 2)$. At concrete state $(1, 0, 0)$, the first value is zero. Using the new second table and stale third table gives $0 + 2 + 1 = 3$, yet a_2 reaches the goal for 2. Allocation feasibility cannot be detached from the tables it certifies. This is a negative control, not an admissible baseline.

Global scaling. Let H be the old SCP sum and define

$$\gamma = \min\left(\{1\} \cup \{c'(a)/c(a) : c(a) > 0\}\right).$$

Then $0 \leq \gamma \leq 1$ and $\gamma c(a) \leq c'(a)$ for every label; an old zero cost imposes no constraint because $c' \geq 0$. Scaling the old consistency inequality yields $\gamma H(s) \leq c'(a) + \gamma H(t)$ and goals remain zero. Hence γH is admissible and consistent. The implementation uses exact rational $\gamma = n/d$ and A* keys $dg + nH$, not floating-point rounding. With all old costs zero the displayed set contains only 1, which is still valid.

Recomputed maximum. Compute each abstraction's exact distances using the full c', then take their maximum. Each projected distance is consistent under c', and taking the maximum preserves that inequality and the goal value. This avoids cost-allocation propagation but can lose additive guidance. The baseline is not an optimal cost partition.

Support-only forward repair. Retain a table only when all its incoming non-self-loop label costs match its saved offer; rebuild it otherwise, and always forward the residual. The same ordered induction proves equality to

a fresh rebuild. It is a deliberately cheaper, less selective exact baseline, not the unsound policy of considering only the original changed action.

Reproducible Checks and Timing Protocol

All experimental inputs are algorithmically generated for this study. No IPC dataset, demonstration, external planner code, training, model API, private data, or hardware service was used. Python 3.13.5 and its standard library were used for the retained run. The production implementation uses arbitrary-precision integer distances and residuals and deliberately rejects floats, Boolean values as costs, negative costs, wrong cost-vector length, and unreachable abstract vertices. The proofs permit finite nonnegative real costs. A separate checker uses exact *Fraction* arithmetic, imports none of the candidate implementation, and exercises fractional ties and reuse certificates without floating-point tolerances; it is regression evidence, not a proof for all reals.

Independent oracle. The oracle performs synchronous Bellman–Ford relaxations, at most $|V| - 1$ rounds, with its own label-major saturation loop and ordered residual subtraction. It does not import the candidate's Dijkstra, saturation, or reuse functions. It shares the input graph representation and synthetic task generators; the comparison is a finite algorithmic cross-check, not a mechanized proof.

For three vertices, labels a, b, and designated goal 2, enumerate all $3^6 = 729$ total deterministic labelled graphs. Exactly 495 have all vertices goal-reachable. For every such graph and all 9×9 old/new vectors in $\{0, 1, 2\}^2$, check candidate distances against the oracle and the reuse answer against exact table equality: 40,095 pairs. The check also recomputes distances under saturated costs. For Lemma 3 it exhaustively checks all 98,415 pairs of four-label vectors in $\{0, 1, 2\}^4$ across all 15 nonempty advancing sets. A separate seeded campaign (seed 91827) creates 120 ordered five-graph sequences, each graph with six states and five labels; a reset-to-goal label ensures finite distances. Twelve arbitrary new integer vectors per sequence give 1,440 sequential mixed updates checked against the independent partition oracle.

For independent and coupled coordinate tasks with $q = 2$, seeds 0–3, and sparse/mixed changes, enumerate every concrete state and transition. An independent concrete Bellman–Ford calculation checks 384 state/update combinations for admissibility, and all their edges for consistency. Path witnesses include $m = 1, 2, 3, 4, 7, 8, 31, 32, 127$; the original pilot additionally included $m = 16, 64$. The amplification checker uses $k = 0, 1, 2, 3, 4, 8, 16, 32, 64$, checking all tables and allocations against the oracle, incidence and support

bounds, exact Fibonacci values, and every nonzero table change. At $k = 64$, the final difference and largest input cost are $F_{67} = 44{,}945{,}570{,}212{,}853$, requiring 46 bits. A separate integer check covers 172,800 four-label inputs, changed coordinates, and pairs of advancing sets for Lemma 4. An independent arithmetic checker verifies all four inequalities of Lemma 5 on 1,849 support-two transitions, the scalar induction inequalities through $k = 1000$, and a deliberately over-approximating reachable envelope through 17 tables; its odd-depth maxima are exactly $1, 2, 3, 5, 8, 13, 21, 34, 55$.

The separate baseline regression explicitly checks the scaled old sum and the recomputed maximum on the same 384 concrete-state updates and every corresponding edge. It also checks all-zero and partly zero old costs on both tiny families, including the empty positive-old-cost ratio set where $\gamma = 1$. These are finite exact-arithmetic correctness checks, not additional timing observations.

The exact-rational checker uses three grids independent of the implementation. Half-integer costs give 1,500 unit-decrease inputs, 22,500 first-stage checks and 337,500 two-stage checks, including 6,084 cases with an old or new minimum tie. A quarter-grid interval over-approximation checks the four P/N inequalities on 153 support-two states and 1,849 transitions. Finally, all 495 goal-reachable three-state/two-label deterministic graphs are crossed with all old/new vectors in $\{0, \frac{1}{2}, 1\}^2$, yielding 40,095 exact-reuse comparisons; 13,689 have a feasible old saturation but a changed table, directly exercising the negative direction of Proposition 8. These finite checks attack the constructions, boundary cases and proof arithmetic; the preceding symbolic proofs establish the arbitrary-size and real-valued claims.

Frozen timing families. Independent tasks have four coordinates in $\{0, \ldots, q-1\}$ with $q \in \{6, 8, 10\}$. For each coordinate there are two identical saturating increment actions, with costs $b, b+4$ where b is sampled uniformly from $\{2, 3, 4, 5\}$, and one decrement of cost 1. Patterns are singletons. Coupled tasks have five coordinates, $q \in \{3, 4, 5\}$, the same individual actions, plus five cyclic adjacent-pair increment actions with costs sampled from $\{2, \ldots, 7\}$. Their patterns are the five adjacent pairs. The four task seeds are 0–3, and each task's pattern order is shuffled once and frozen. Coordinate maps clamp to the legal domain, have no preconditions, and exactly commute with projection. Goals set all coordinates to $q - 1$ and starts are all zero.

The update generator has seed $10000 +$ task seed, independent of task construction. It uniformly chooses one label and decreases its cost by a uniform integer from 1 to 6, clamped at zero. The mixed condition additionally increases a distinct uniformly chosen label by a uniform integer from 1 to

4. It does not select labels by cost slack, effect on a table, search difficulty, or measured outcome. Each family therefore contributes $3 \times 4 \times 2 = 24$ updates. Path relays use $m \in \{6, 10, 14\}$, old costs 4 and 8, and decreases $\delta \in \{1, 2, 3, 4\}$, contributing 12 updates. The 60-case timing set was fixed before its execution. Amplification witnesses are proof checks, not post-selected additions to this timing set.

Measurement. Each case is run three times per method, with the five-method execution order cyclically rotated by case index plus repeat index. One worker is used. Every run has fresh A* state, identical deterministic tie order $(f, -g, \text{state})$, and permits reopening and zero-cost actions. The goal is not counted as an expanded state. Search is limited to 100,000 expansions and an 8-second cooperative time check every 128 expansions; the clean reproduction additionally has a 40-second hard timeout for each child command. The actual largest concrete space is 16,384 states, largest collection has 125 abstract entries, and maximum action count is 20.

The timed update and search intervals exclude graph construction, original tables, full reference recomputation for equality checks, and old/new comparisons used only for accounting. Every method pays for creation of its new heuristic evaluator. Initial setup is recorded separately in every raw case. Distance, saturation and certificate scans are instrumented in the measured routines. All 900 primary timed searches finish, with no omitted cases; exact reuse, support-only and full rebuilding have identical tables, allocations, and expansions. All five methods return the same optimal cost per case. For large timing cases, this agreement is not itself an independent concrete-state oracle; the independent concrete oracle is restricted to the tiny cases above.

The campaign driver saves an unsuccessful case before exiting nonzero. Summaries retain incomplete runs and failure counts rather than filtering them, and their command exits nonzero if any search is incomplete. The table generator refuses such summaries, so its all-solved caption is conditional on zero failures. Case specifications, unique method–repeat records, solution-cost agreement, and deterministic successful-repeat counters are checked before aggregation. These reporting checks do not replace the independent distance oracle or establish a search-completion guarantee on slower hosts.

Family	Method	Cases	Written (%)	Update (ms)	Update+A* (ms)	Ratio to full	Expansions
Independent	Full SCP	24	100.0	2.513	6.71	1.000	672
	Exact reuse	24	21.9	0.934	5.12	0.763	672
	Support only	24	37.5	0.913	4.91	0.731	672
	Scaled old	24	0.0	0.704	633.47	94.405	123,028
	New maximum	24	100.0	0.836	635.54	94.712	121,576
Coupled	Full SCP	24	100.0	8.113	32.27	1.000	2,272
	Exact reuse	24	22.5	2.808	27.58	0.855	2,272
	Support only	24	75.0	5.579	29.90	0.927	2,272
	Scaled old	24	0.0	0.627	240.43	7.451	27,839
	New maximum	24	100.0	2.552	152.88	4.738	15,895
Relay	Full SCP	12	100.0	1.019	1.97	1.000	66
	Exact reuse	12	100.0	1.117	2.03	1.029	66
	Support only	12	100.0	0.972	1.88	0.956	66
	Scaled old	12	0.0	0.238	105.09	53.296	13,404
	New maximum	12	100.0	0.291	102.10	51.783	12,585

Table 1: Primary synthetic campaign by family. Times sum per-instance medians; repetitions are not independent tasks. Ratios use the summed update-plus-search times within a family. Written percentages are weighted by table size. All methods solve every case.

The table sums each instance's median time over its three repeats. Expansion totals count each instance once, not three times. Rewritten entries count an entire materialized distance table, including its goal, when that table is rebuilt. They do not count all reads, cached allocation metadata, or temporary structures; those costs remain in measured time and scan counters. Exact reuse writes 28.5% of the entries overall, but writes all relay tables and is slightly slower there. The independent-coordinate family intentionally exposes additive guidance; the scaled and maximum baselines' large expansion counts here should not be generalized to a production benchmark collection. The paired per-case minimum and maximum total times, changed entries, certificate entries, and all raw runs are retained, not just the aggregate means.

Three repeats characterize local timing variation, not confidence about a population of planning tasks. Small Python PDBs and microsecond update intervals are particularly sensitive to interpreter and scheduling overhead. Clean-extraction reproduction checks deterministic outputs separately from timings; the latter are allowed to change. In the two retained clean reproductions, the overall exact/full total-time ratios were 0.808 and 0.774 (primary: 0.848), while the relay ratios were 1.082 and 1.019 (primary: 1.029). All deterministic fields of all 900 runs matched in both reproductions. Neither the oracle counts nor the synthetic speed ratios establish production-planner competitiveness.

Relation to Existing Work and Artifact Boundaries

SCP's recurrence and admissibility are established [53]. Existing decomposition methods optimize partitions [34]; exact LP sensitivity reuses solutions across state-dependent queries [19]. Monotonic variants address how adding or refining abstractions affects SCP [40]. Incremental search, including Lifelong Planning A*, reuses search information after graph changes [30]. None of these comparisons licenses a claim that goal-path optimality certificates or shortest-path reuse are new. The proposed contribution is the explicit bounded-incidence cost-decrease construction and its consequence for which tables must change under a fixed ordered partition.

The source repository includes the complete generators, oracle, tests, frozen inputs, raw outputs, summary code and proof source. Its README gives one fresh-output reproduction command. The article's Code Availability section links this repository. General signed allocations, dead-end arithmetic, changes to patterns/order, multiple-order maximization, dynamic maintenance within a partially rebuilt PDB, production PDDL integration,

and policies for choosing between exact reuse and full rebuilding are outside the implemented model. They are not hidden prerequisites for the claims proved here.

Self-Loop Elision

A self-loop contributes $h(u) - h(u) = 0$ to the zero-initialized saturation maximum. Preindexing non-self edges preserves allocations, including zero for self-loop-only labels; ordered residual induction preserves tables. Full transitions remain available.

Windows CPython uses 60 frozen updates, eleven rotating pairs of four arms (full and exact updates, each with all-edge and elided kernels), and sixteen update/evaluator calls per arm. Indexing and old/reference tables are untimed. All 2,640 A* runs match cost, expansions and generated nodes; tables and allocations agree. Tests pass for 2,000 random saturation vectors, 40,095 exhaustive reuse cases and 1,440 Bellman–Ford sequential comparisons. The artifact retains raw pairs, edge counts and CPU affinity. Median ratios span 1.37–1.98 for full updates, 1.00–1.67 for exact updates. These kernel timings exclude search and cannot resolve differences near one; Linux all-edge timings remain separate.

These measurements predate the current all-edge prepared-cost implementation; they describe the retained historical kernels, not a new timing of the current code.

Related Work

This section relates the fixed-order residual model to shortest-path, abstraction, and cost-partitioning methods. The accompanying literature matrix distinguishes publication identity from the particular source passages examined.

Shortest Paths, Heuristic Search, and Planning Context

The underlying distance computations descend from shortest-path dynamic programming and label-setting algorithms [1, 7]. Admissible best-first search and its standard textbook treatment establish the search-side interpretation of lower bounds [13, 32]. Classical planning as heuristic search and the Fast Downward architecture place those ideas in the planning setting [3, 15]. State-equation, landmark, abstraction, and potential heuristics show that cost-optimal planning already contains several families of admissible lower bounds and optimization views [2, 24, 16, 57]. These works motivate

the setting, but they do not study how one externally changed action cost is transformed by a fixed sequence of saturation subtractions.

Pattern Databases and Abstraction Heuristics

Pattern databases originated as stored exact distances in an abstraction [5, 31]; automatic construction brought them into optimal planning [9, 10]. Additive and maximum combinations clarify admissible aggregation [11, 22], including multiple-pattern and general abstraction settings [14, 63]. Optimal abstraction composition and planning-specific PDB selection make cost allocation and pattern interaction explicit [25, 35].

Cartesian abstraction refinement supplies another route to exact abstract distances, including diverse additive collections and the later full treatment [49, 50, 51]. Merge-and-shrink develops a complementary abstraction family and analyzes label reduction and merge choices [17, 58, 59]. Later work on refinement, factored tasks, transition generation, conditional effects, and abstraction merging broadens the construction techniques available to a planner [61, 4, 47, 38, 41]. Planner ablations and the dissertation-level synthesis document how Cartesian abstractions and saturated partitions operate in complete systems [46, 43]. Our constructions intentionally use the smallest possible two-state abstractions, so the result cannot be attributed to a difficult abstraction-building procedure.

Cost Partitioning and Saturated Cost Partitioning

General operator-cost partitioning, LP-based heuristics, state-dependent partitions, and Lagrangian formulations establish that allocating action cost among abstractions is an optimization problem with many admissible solutions [33, 36, 26, 37]. The development of saturated cost partitioning includes comparative evaluation, order optimization, tightening toward optimal partitions, pattern selection, subset saturation, and the complete journal treatment [53, 42, 54, 44, 52, 55]. We inherit the ordered recurrence and its admissibility argument from this line; neither is renamed as a contribution here.

Several nearby methods change what is optimized or represented. Online SCP changes which stored orders are evaluated; transition cost partitioning allocates at transition rather than label granularity; saturated post-hoc optimization and Dantzig–Wolfe decomposition optimize combinations rather than replaying one frozen ordered recurrence [45, 8, 56, 34]. Sensitivity analysis reuses LP solutions, probabilistic planning changes the task semantics, perfect SCP compresses order behavior, monotonic variants study adding

or refining abstractions, and cost partitioning for sequence alignment transfers the technique to another domain [20, 27, 19, 21, 40, 60]. These are the strongest semantic neighbors. None, in the reviewed formulation, proves a bounded-incidence external-cost cascade for a fixed abstraction set and order or gives the table-bound exact-reuse rule analyzed here.

Dynamic and Parametric Shortest Paths

Incremental search has also been used inside Cartesian abstraction refinement [48]. Lifelong Planning A*, D* Lite, focused D*, and later navigation/replanning analyses reuse shortest-path information as one graph changes [30, 28, 62, 29, 18]. General dynamic shortest-path algorithms maintain one graph under updates, while parametric shortest-path work studies structured variation of edge weights [39, 12, 6, 23, 64]. Such an algorithm may accelerate a single rebuilt abstraction. It does not by itself decide which later abstraction receives which new residual costs, because those costs are outputs of earlier saturation stages. This distinction is why the relay remains a counterexample even when every individual PDB is trivial.

Comparison of Models

Table 2 states the collision test used to keep the claim narrow. The negative-result route is important: incremental repair is not uniformly faster than recomputation, so the artifact reports relay cases where its exact certificate overhead loses rather than filtering them out [18]. Likewise, a familiar shortest-path optimality certificate would not be enough by itself. The retained contribution is the composition result: a sparse external update can travel and amplify through the residual pipeline, and a cached table must remain paired with the allocation that preserves it.

The comparisons separate fixed-order residual interaction from single-graph distance maintenance, order selection, and partition optimization. The Fibonacci construction, relay, and table/allocation consistency results address that interaction.

Neighbor	Already established	Boundary retained here
SCP and order methods [53, 55, 42]	Ordered saturation, admissibility, and the importance of abstraction order.	External action costs change between searches while graphs and one order stay fixed; the question is propagation through that fixed recurrence.
Optimization and sensitivity [34, 20, 19]	Partition optimization and exact reuse regions for LP-based formulations.	Repair retains saved exact PDB values and their saturations.
Online and perfect SCP [45, 21]	Efficient evaluation or representation of many useful abstraction orders.	The abstraction sequence and its single order remain fixed.
Monotonic and transition variants [8, 40]	Finer allocation units and behavior when abstraction collections change.	Labels remain shared across projected transitions and the abstraction collection is unchanged.
Dynamic shortest paths [39, 30, 48]	Reuse of distances after changes in one graph.	Such reuse is an interchangeable within-table subroutine; it does not remove inter-table residual dependence.
This work	Uses established shortest-path distances, PDB construction, and the SCP recurrence.	A tight degree-three Fibonacci family, a degree-two arbitrary relay, and exact ordered repair with explicit stale-value and zero-cycle controls.

Table 2: Related methods and the fixed-order residual model.

References

[1] Bellman, R.E., 1958. On a routing problem. Quarterly of Applied Mathematics 16, 87–90.

[2] Bonet, B., 2013. An admissible heuristic for SAS+ planning obtained from the state equation, in: Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence, pp. 2268–2274.

[3] Bonet, B., Geffner, H., 2001. Planning as heuristic search. Artificial Intelligence 129, 5–33.

[4] Büchner, C., Ferber, P., Seipp, J., Helmert, M., 2024. Abstraction heuristics for factored tasks, in: Proceedings of the Thirty-Fourth International Conference on Automated Planning and Scheduling, pp. 40–49.

[5] Culberson, J.C., Schaeffer, J., 1998. Pattern databases. Computational Intelligence 14, 318–334.

[6] Demetrescu, C., Italiano, G.F., 2004. A new approach to dynamic all pairs shortest paths. Journal of the ACM 51, 968–992.

[7] Dijkstra, E.W., 1959. A note on two problems in connexion with graphs. Numerische Mathematik 1, 269–271.

[8] Drexler, D., Seipp, J., Speck, D., 2021. Subset-saturated transition cost partitioning, in: Proceedings of the Thirty-First International Conference on Automated Planning and Scheduling, pp. 131–139.

[9] Edelkamp, S., 2001. Planning with pattern databases, in: Proceedings of the Sixth European Conference on Planning, pp. 84–90. URL: https://cdn.aaai.org/ocs/7280/7280-37829-1-PB.pdf.

[10] Edelkamp, S., 2007. Automated creation of pattern database search heuristics, in: Model Checking and Artificial Intelligence, Springer. pp. 35–50. doi:10.1007/978-3-540-74128-2_3.

[11] Felner, A., Korf, R.E., Hanan, S., 2004. Additive pattern database heuristics. Journal of Artificial Intelligence Research 22, 279–318.

[12] Frigioni, D., Marchetti-Spaccamela, A., Nanni, U., 2000. Fully dynamic algorithms for maintaining shortest paths trees. Journal of Algorithms 34, 251–281. doi:10.1006/jagm.1999.1048.

[13] Hart, P.E., Nilsson, N.J., Raphael, B., 1968. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics 4, 100–107.

[14] Haslum, P., Botea, A., Helmert, M., Bonet, B., Koenig, S., 2007. Domain-independent construction of pattern database heuristics for cost-optimal planning, in: Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence, pp. 1007–1012. URL: https://cdn.aaai.org/AAAI/2007/AAAI07-160.pdf.

[15] Helmert, M., 2006. The fast downward planning system. Journal of Artificial Intelligence Research 26, 191–246.

[16] Helmert, M., Domshlak, C., 2009. Landmarks, critical paths and abstractions: What's the difference anyway?, in: Proceedings of the Nineteenth International Conference on Automated Planning and Scheduling, pp. 162–169.

[17] Helmert, M., Haslum, P., Hoffmann, J., Nissim, R., 2014. Merge-and-shrink abstraction: A method for generating lower bounds in factored state spaces. Journal of the ACM 61, 16:1–16:63.

[18] Hernández, C., Baier, J., Uras, T., Koenig, S., 2012. Position paper: Incremental search algorithms considered poorly understood, in: Proceedings of the Fifth Annual Symposium on Combinatorial Search, pp. 159–161. doi:10.1609/socs.v3i1.18268.

[19] Höft, P., Speck, D., Pommerening, F., Seipp, J., 2024. Versatile cost partitioning with exact sensitivity analysis, in: Proceedings of the Thirty-Fourth International Conference on Automated Planning and Scheduling, pp. 276–280.

[20] Höft, P., Speck, D., Seipp, J., 2023. Sensitivity analysis for saturated post-hoc optimization in classical planning, in: Proceedings of the Twenty-Sixth European Conference on Artificial Intelligence, pp. 1044–1051.

[21] Höft, P., Speck, D., Seipp, J., 2025. Representing perfect saturated cost partitioning heuristics in classical planning, in: Proceedings of the Twenty-Second International Conference on Principles of Knowledge Representation and Reasoning, pp. 821–831.

[22] Holte, R.C., Felner, A., Newton, J., Meshulam, R., Furcy, D., 2006. Maximizing over multiple pattern databases speeds up heuristic search. Artificial Intelligence 170, 1123–1136. doi:10.1016/j.artint.2006.09.002.

[23] Karp, R.M., Orlin, J.B., 1981. Parametric shortest path algorithms with an application to cyclic staffing. Discrete Applied Mathematics 3, 37–45. doi:10.1016/0166-218X(81)90026-3.

[24] Karpas, E., Domshlak, C., 2009. Cost-optimal planning with landmarks, in: Proceedings of the Twenty-First International Joint Conference on Artificial Intelligence, pp. 1728–1733.

[25] Katz, M., Domshlak, C., 2008. Optimal additive composition of abstraction-based admissible heuristics, in: Proceedings of the Eighteenth International Conference on Automated Planning and Scheduling, AAAI Press. pp. 174–181. URL: https://ctpelok77.github.io/papers/icaps08b.pdf.

[26] Keller, T., Pommerening, F., Seipp, J., Geißer, F., Mattmüller, R., 2016. State-dependent cost partitionings for cartesian abstractions in classical planning, in: Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, pp. 3161–3169.

[27] Klößner, T., Seipp, J., Steinmetz, M., 2023. Cartesian abstractions and saturated cost partitioning in probabilistic planning, in: Proceedings of the Twenty-Sixth European Conference on Artificial Intelligence, pp. 1272–1279.

[28] Koenig, S., Likhachev, M., 2002. D* Lite, in: Proceedings of the Eighteenth National Conference on Artificial Intelligence, pp. 476–483.

[29] Koenig, S., Likhachev, M., 2005. Fast replanning for navigation in unknown terrain. IEEE Transactions on Robotics 21, 354–363.

[30] Koenig, S., Likhachev, M., Furcy, D., 2004. Lifelong planning A*. Artificial Intelligence 155, 93–146.

[31] Korf, R.E., 1997. Finding optimal solutions to Rubik's Cube using pattern databases, in: Proceedings of the Fourteenth National Conference on Artificial Intelligence, pp. 700–705.

[32] Pearl, J., 1984. Heuristics: Intelligent Search Strategies for Computer Problem Solving. Addison-Wesley.

[33] Pommerening, F., Helmert, M., Röger, G., Seipp, J., 2015. From non-negative to general operator cost partitioning, in: Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, pp. 3335–3341. doi:10.1609/aaai.v29i1.9668.

[34] Pommerening, F., Keller, T., Halasi, V., Seipp, J., Sievers, S., Helmert, M., 2021. Dantzig–wolfe decomposition for cost partitioning, in: Proceedings of the Thirty-First International Conference on Automated Planning and Scheduling, pp. 271–280.

[35] Pommerening, F., Röger, G., Helmert, M., 2013. Getting the most out of pattern databases for classical planning, in: Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence, pp. 2357–2364.

[36] Pommerening, F., Röger, G., Helmert, M., Bonet, B., 2014. LP-based heuristics for cost-optimal planning, in: Proceedings of the Twenty-

Fourth International Conference on Automated Planning and Scheduling, pp. 226–234.

[37] Pommerening, F., Röger, G., Helmert, M., Cambazard, H., Rousseau, L.M., Salvagnin, D., 2019. Lagrangian decomposition for optimal cost partitioning, in: Proceedings of the Twenty-Ninth International Conference on Automated Planning and Scheduling, pp. 338–347.

[38] Pozo, M., Seipp, J., 2025. Abstraction heuristics for classical planning tasks with conditional effects, in: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, pp. 8608–8616.

[39] Ramalingam, G., Reps, T., 1996. An incremental algorithm for a generalization of the shortest-path problem. Journal of Algorithms 21, 267–305. doi:10.1006/jagm.1996.0046.

[40] Salerno, M., Fuentetaja, R., Seipp, J., 2026. Monotonic variants of saturated cost partitioning, in: Proceedings of the Thirty-Sixth International Conference on Automated Planning and Scheduling, pp. 415–419.

[41] Salerno, M., Fuentetaja, R., Speck, D., Seipp, J., 2025. Merging cartesian abstractions for classical planning, in: Proceedings of the Twenty-Eighth European Conference on Artificial Intelligence, pp. 4766–4773.

[42] Seipp, J., 2017. Better orders for saturated cost partitioning in optimal classical planning, in: Proceedings of the Tenth Annual Symposium on Combinatorial Search, pp. 149–153.

[43] Seipp, J., 2018. Counterexample-Guided Cartesian Abstraction Refinement and Saturated Cost Partitioning for Optimal Classical Planning. Ph.D. thesis. University of Basel.

[44] Seipp, J., 2019. Pattern selection for optimal classical planning with saturated cost partitioning, in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, pp. 5621–5627.

[45] Seipp, J., 2021. Online saturated cost partitioning for classical planning, in: Proceedings of the Thirty-First International Conference on Automated Planning and Scheduling, pp. 317–321.

[46] Seipp, J., 2024a. Dissecting scorpion: Ablation study of an optimal classical planner, in: Proceedings of the Twenty-Seventh European Conference on Artificial Intelligence, pp. 39–42.

[47] Seipp, J., 2024b. Efficiently computing transitions in cartesian abstractions, in: Proceedings of the Thirty-Fourth International Conference on Automated Planning and Scheduling, pp. 509–513.

[48] Seipp, J., von Allmen, S., Helmert, M., 2020a. Incremental search for counterexample-guided cartesian abstraction refinement, in: Proceedings of the Thirtieth International Conference on Automated Planning and Scheduling, pp. 244–248.

[49] Seipp, J., Helmert, M., 2013. Counterexample-guided cartesian abstraction refinement, in: Proceedings of the Twenty-Third International Conference on Automated Planning and Scheduling, pp. 347–351.

[50] Seipp, J., Helmert, M., 2014. Diverse and additive cartesian abstraction heuristics, in: Proceedings of the Twenty-Fourth International Conference on Automated Planning and Scheduling, pp. 289–297.

[51] Seipp, J., Helmert, M., 2018. Counterexample-guided cartesian abstraction refinement for classical planning. Journal of Artificial Intelligence Research 62, 535–577. doi:10.1613/jair.1.11217.

[52] Seipp, J., Helmert, M., 2019. Subset-saturated cost partitioning for optimal classical planning, in: Proceedings of the Twenty-Ninth International Conference on Automated Planning and Scheduling, pp. 391–400.

[53] Seipp, J., Keller, T., Helmert, M., 2017a. A comparison of cost partitioning algorithms for optimal classical planning, in: Proceedings of the Twenty-Seventh International Conference on Automated Planning and Scheduling, pp. 259–268.

[54] Seipp, J., Keller, T., Helmert, M., 2017b. Narrowing the gap between saturated and optimal cost partitioning for classical planning, in: Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, pp. 3651–3657.

[55] Seipp, J., Keller, T., Helmert, M., 2020b. Saturated cost partitioning for optimal classical planning. Journal of Artificial Intelligence Research 67, 129–167.

[56] Seipp, J., Keller, T., Helmert, M., 2021. Saturated post-hoc optimization for classical planning, in: Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence, pp. 11947–11953.

[57] Seipp, J., Pommerening, F., Helmert, M., 2015. New optimization functions for potential heuristics, in: Proceedings of the Twenty-Fifth International Conference on Automated Planning and Scheduling, pp. 193–201.

[58] Sievers, S., Wehrle, M., Helmert, M., 2014. Generalized label reduction for merge-and-shrink heuristics, in: Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence, pp. 2358–2366.

[59] Sievers, S., Wehrle, M., Helmert, M., 2016. An analysis of merge strategies for merge-and-shrink heuristics, in: Proceedings of the Twenty-Sixth International Conference on Automated Planning and Scheduling, pp. 294–298.

[60] Skjelnes, M., Gnad, D., Seipp, J., 2024. Cost partitioning for multiple sequence alignment, in: Proceedings of the Twenty-Seventh European Conference on Artificial Intelligence, pp. 4224–4231.

[61] Speck, D., Seipp, J., 2022. New refinement strategies for cartesian abstractions, in: Proceedings of the Thirty-Second International Conference on Automated Planning and Scheduling, pp. 348–352.

[62] Stentz, A., 1995. The focussed D* algorithm for real-time replanning, in: Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence, pp. 1652–1659.

[63] Yang, F., Culberson, J.C., Holte, R., Zahavi, U., Felner, A., 2008. A general theory of additive state space abstractions. Journal of Artificial Intelligence Research 32, 631–662. doi:10.1613/jair.2486.

[64] Young, N.E., Tarjan, R.E., Orlin, J.B., 1991. Faster parametric shortest path and minimum-balance algorithms. Networks 21, 205–221.